SOFTWARE ENGINEERING LAB

LAB 24

SLO, SLI, dan Error Budget di Go

Implementasi time-bucketed sliding window, evaluasi error budget matematis, dan multi-window multi-burn-rate alerting berbasis prinsip Google SRE.



Masalah Pengukuran Keandalan Tradisional

Metrik server CPU dan memori tidak mencerminkan pengalaman nyata pengguna.

MOTIVASI

Tim keandalan kerap mengukur kesehatan sistem memakai CPU dan RAM. Layanan dengan utilisasi 15% bisa mengalami failure total akibat error HTTP 500, sementara server sibuk 90% dapat melayani 100% transaksi secara sukses. Metrik infrastruktur menyamarkan kegagalan pengguna.

BATAS LAB

Lab ini mengimplementasikan SLI good/total ratio, sliding window tracker dengan bucketing waktu, evaluasi error budget matematis, dan dual-window burn rate alert engine di Go murni tanpa dependensi eksternal.

Infrastructure metrics measure server health; SLIs measure user pain.

Mental Model: Keandalan Bukan 100%

Menargetkan uptime 100% adalah ilusi teknis dan pemborosan biaya rekayasa.

PRINSIP UTAMA

Keandalan sempurna (100%) tidak realistis karena jaringan seluler, ISP, dan perangkat pengguna memiliki batas kegagalan tersendiri. Menambah sembilan (nine) ketersediaan dari 99.9% ke 99.99% melipatgandakan biaya infrastruktur secara eksponensial.

ERROR BUDGET = KUOTA INOVASI

Error Budget didefinisikan sebagai (1.0 - SLO). Selisih ini adalah kuota kegagalan yang sah digunakan tim produk untuk merilis fitur baru, eksperimen arsitektur, dan pembaruan infrastruktur tanpa rasa takut.

Reliability is not 100%; the error budget is an explicit allocation for innovation.

Taksonomi: SLI, SLO, dan SLA

Membedakan indikator teknis, target internal, dan kontrak hukum bisnis.

SLI & SLO (INTERNAL TEKNIS)

• SLI (Service Level Indicator): Rasio kuantitatif kejadian sukses terhadap total kejadian valid. • SLO (Service Level Objective): Target batas internal untuk SLI (misal 99.9% sukses selama 30 hari) guna mengarahkan prioritas engineering.

SLA (KONTRAK HUKUM)

• SLA (Service Level Agreement): Perjanjian legal dengan pelanggan eksternal yang mencakup konsekuensi finansial atau penalti refund. SLO internal selalu disetel lebih ketat dibanding SLA eksternal.

SLI measures reality; SLO guides internal engineering; SLA defines contractual penalties.

Formulasi Matematis Error Budget

Kalkulasi sisa kuota kesalahan absolut berbasis total volume transaksi.

RUMUS PERHITUNGAN

Total kuota kegagalan dihitung secara matematis: • totalErrorBudget = (1.0 - TargetSLO) * TotalRequests • budgetRemaining = totalErrorBudget - BadRequests Jika budgetRemaining <= 0 saat ada trafik, deployment freeze aktif.

KEBIJAKAN RELEASE FREEZE

Sisa error budget menjadi penentu otomatis gerbang CI/CD. Ketika budget negatif, variabel CanDeploy berubah menjadi false. Semua perubahan berisiko dihentikan kecuali hotfix perbaikan keandalan dan patching keamanan P0.

When the error budget reaches zero, non-emergency feature deployments must freeze.

Agregasi Time-Bucketed Sliding Window

Menghindari ledakan memori dengan pengelompokan interval waktu tetap.

TANTANGAN IN-MEMORY

Menyimpan jutaan event request secara individual dalam memory array memicu overhead alokasi pointer dan membebani Go Garbage Collector. Pengukuran jendela berjalan membutuhkan struktur data yang kompak.

STRUKTUR DATA BUCKET

WindowTracker mengelompokkan request ke dalam bucket interval waktu (misal 10 detik atau 1 menit). Setiap bucket hanya menyimpan pencacah numerik: StartTime, TotalCount, GoodCount, dan BadCount.

Fixed-size time bucketing eliminates GC overhead in high-throughput sliding windows.

Implementasi Thread-Safe WindowTracker

Sinkronisasi mutlak mutex, pemangkasan bucket usang, dan truncation waktu.

METODE RECORD()

Setiap event masuk mengunci sync.RWMutex secara eksklusif. Tracker memanggil evict() untuk menghapus bucket di luar jendela, memotong timestamp via Truncate(bucketSize), lalu menginkremen pencacah yang relevan.

METODE SUMMARY()

Pembacaan ringkasan mengunci tracker dengan RLock(). Sistem menjumlahkan TotalCount, GoodCount, dan BadCount dari seluruh bucket yang masih valid di dalam batas windowSize secara atomik.

Synchronized sliding windows guarantee thread-safe eviction and zero metric loss.

Evaluator SLO dan Edge Case Zero Traffic

Mencegah deployment freeze palsu saat inisialisasi sistem baru.

KONDISI VOLUME NOL

Ketika service baru dinyalakan atau tidak ada request masuk (total == 0), pembagian Good / Total akan menghasilkan NaN atau 0%. Jika tidak ditangani, hal ini memicu deployment freeze yang keliru.

LOGIKA HANDLER DEFENSIVE

Evaluator secara eksplisit mendeteksi total == 0 dan mengembalikan Status dengan CurrentSLI: 1.0 dan CanDeploy: true. Deployment hanya diblokir jika kegagalan nyata terbukti saat ada lalu lintas.

Defensive zero-traffic guards prevent false deployment freezes during service bootstrap.

Uji Otomatis: Out-of-Order Timestamps

Ketahanan agregator terhadap paket jaringan yang tiba tidak berurutan.

SKENARIO UJI

TestOutOfOrderTimestamps menyuntikkan event dengan urutan waktu acak: T-10m, T-2m, dan T-5m. Tracker harus menempatkan event ke bucket yang tepat tanpa merusak urutan slice.

HASIL VERIFIKASI

Sistem secara otomatis mengurutkan bucket dan membuang event yang melewati batas kadaluarsa. Total agregasi tetap konsisten dan tepat merefleksikan event di dalam batas aktif.

Robust metric aggregators must handle out-of-order network events deterministically.

Kelemahan Alerting Satu Jendela (Single Window)

Dilema operasional antara alarm palsu dan keterlambatan deteksi insiden.

JENDELA PENDEK (5 MENIT)

Sangat responsif terhadap insiden, tetapi menghasilkan false alarm tinggi saat ada lonjakan galat sesaat (transient spike) akibat restart pod atau timeout jaringan 10 detik.

JENDELA PANJANG (24 JAM)

Tahan terhadap noise sesaat, namun Mean Time to Detect (MTTD) menjadi sangat lambat. Ketika sistem mengalami pemadaman total 100%, alarm baru berbunyi berjam-jam kemudian.

Single-window alerting forces an unresolvable trade-off between noise and detection delay.

Konsep Burn Rate: Laju Konsumsi Budget

Mengukur percepatan penghabisan kuota kesalahan relatif terhadap target.

DEFINISI MATEMATIS

Burn rate = actual_error_rate / allowed_error_rate. • SLO 99.9% mengizinkan error rate 0.1%. • Jika error rate aktual adalah 2.0%, maka Burn Rate = 2.0% / 0.1% = 20× lebih cepat dari toleransi normal.

ARTI OPERASIONAL

Burn rate 1.0x menghabiskan 100% budget tepat di akhir periode 30 hari. Burn rate 14.4x menghabiskan 2% budget hanya dalam 1 jam, membutuhkan tindakan pager darurat seketika.

Burn rate quantifies how fast an incident consumes the budget relative to its allowed rate.

Multi-Window Multi-Burn-Rate Alerting

Mekanisme eliminasi false alarm berbasis verifikasi simultan dua jendela.

DORONGAN ARSITEKTUR

Google SRE memecahkan dilema alerting dengan memeriksa dua jendela sekaligus: • Jendela Pendek (Short Window): Memverifikasi keparahan laju saat ini. • Jendela Panjang (Long Window): Memverifikasi akumulasi durasi konsumsi.

SYARAT PEMICUAN LOGIS

Peringatan HANYA dipicu jika ShortBurnRate >= Threshold DAN LongBurnRate >= Threshold. Jika spike hanya terjadi di jendela pendek, sistem mengabaikannya sebagai noise.

Dual-window validation eliminates transient noise while catching sustained outages fast.

Hierarki Alert: Page Fast vs Ticket Slow

Menyelaraskan tingkat keparahan alarm dengan tindakan tim rekayasa.

PAGE FAST (KRITIS)

• Burn Rate: 14.4× (menghabiskan 2% budget dalam 1 jam). • Jendela: 5 Menit (Short) & 1 Jam (Long). • Tindakan: Bangunkan insinyur on-call seketika untuk intervensi mitigasi darurat.

TICKET SLOW (NON-KRITIS)

• Burn Rate: 6.0× (menghabiskan 5% budget dalam 6 jam). • Jendela: 30 Menit (Short) & 6 Jam (Long). • Tindakan: Buat tiket issue pelacakan backlog tanpa mengganggu tidur on-call.

Page only on catastrophic burns; create automated tickets for slow budget leaks.

Implementasi AlertEngine di Go

Evaluasi dual-tracker konkuren dan pembentukan record alert terstruktur.

STRUKTUR ENGINE

AlertEngine menyimpan referensi ke shortTracker dan longTracker. Fungsi Check() mengambil ringkasan metrik dari kedua tracker di bawah perlindungan RLock.

EVALUASI RULE BERSIH

Engine menghitung sBurn dan lBurn secara independen. Setiap rule yang terlewati di kedua sisi dikonversi menjadi struct Alert berisi RuleName, Severity, dan laju pembakaran aktual.

Isolated dual-tracker engines deliver clean, deterministic alert evaluation in Go.

Uji Otomatis: Supresi Spike Transien

Bukti empiris pencegahan alert palsu saat lonjakan galat sesaat.

KONDISI PENGUJIAN

TestAlertEngineBurnRate menguji spike ekstrim pada jendela pendek dengan burn rate mencapai 100x, sementara jendela panjang tetap bersih (burn rate 0.1x).

HASIL EVALUASI SUITE

Engine mengembalikan 0 alert aktif. Bukti otomatis memverifikasi bahwa lonjakan sesaat tidak memicu false positive. Alarm hanya terpicu saat kedua window melampaui batas.

Empirical tests verify 100% transient noise suppression under dual-window alerting.

Ketahanan Konkurensi: 20 Goroutine Bebas Race

Verifikasi integritas memori di bawah beban paralel tinggi via go test -race.

BEBAN UJI KONKURENSI

TestConcurrencyMetrics mengeksekusi 20 goroutine yang secara simultan menyuntikkan 100 event per goroutine (total 2.000 event) ke dalam WindowTracker bersama.

HASIL RACE DETECTOR

Semua 2.000 event tercatat secara presisi tanpa ada counter yang hilang. Flag -race melaporkan 0 data race, membuktikan keamanan sinkronisasi sync.RWMutex.

Thread-safe metric collectors guarantee total data integrity under concurrent load.

Orkestrasi Demo: Fase 1 & 2 (Baseline & Insiden)

Simulasi transisi dari operasional sehat ke kondisi error budget habis.

FASE 1: TRAFIK SEHAT

1.000 request disuntikkan dengan 100% sukses. Evaluasi menghasilkan: • SLI: 100.00% | Budget Tersisa: +1.00 error • Status Deploy: ALLOWED (CanDeploy = true).

FASE 2: INJEKSI INSIDEN

10 error disuntikkan dalam 100 request (10% burst). Akumulasi: • Total: 1.100 request | Bad: 10 | SLI: 99.09% • Budget Tersisa: -8.90 error | CanDeploy: false.

A 10% error burst exhausts the budget immediately, triggering automated release freeze.

Orkestrasi Demo: Fase 3 & 4 (Alert & Kritisitas)

Pemicuan alarm nyata dan diferensiasi target antar endpoint bisnis.

FASE 3: ALERT TERPICU

Engine mendeteksi short burn 9.09x dan long burn 9.09x. Nilai ini melampaui threshold 6.0x, memicu alarm [TICKET] Slow Burn Alert secara otomatis.

FASE 4: TIERED SLO

Komparasi endpoint kritis vs reguler: • Payment Service (SLO 99.9%): Budget -8.90 (Freeze) • Reports Service (SLO 95.0%): Toleransi 5.0% jauh lebih longgar, menyerap fluktuasi trafik ringan.

Tiered SLO thresholds match operational alerts to core business criticality.

Matriks Verifikasi Pengujian Otomatis

Seluruh suite pengujian lolos tanpa kegagalan matematis dan sinkronisasi.

RINGKASAN SUITE TEST

• TestMetricsWindowTracker: Agregasi bucket presisi (PASS) • TestOutOfOrderTimestamps: Penanganan waktu acak (PASS) • TestEvaluatorZeroTraffic: Guard trafik nol (PASS) • TestSLOEvaluator: Freeze budget habis (PASS)

HASIL ALERT & RACE

• TestAlertEngineBurnRate: Dual-window alert & spike suppression (PASS) • TestConcurrencyMetrics: 20 worker paralel bebas data race (PASS) Total durasi: 2.904s di bawah go test -race.

All 6 automated test assertions pass with zero data races and verified math.

Prosedur Pemulihan (Incident Recovery)

Langkah terstruktur tim rekayasa saat error budget berada di zona merah.

TINDAKAN REKAYASA SEGERA

1. Hentikan rilis fitur baru pada repositori terkait. 2. Alihkan fokus sprint engineering ke investigasi akar masalah (root cause analysis). 3. Terapkan mitigasi hotfix berisiko rendah atau rollback versi.

PEMULIHAN KUOTA BUDGET

Seiring waktu berjalan, bucket berisi event error akan terdorong keluar dari jendela geser (eviction). Begitu rasio kegagalan turun di bawah batas, budget kembali positif dan deploy dibuka.

Budget recovery occurs naturally as rolling windows evict historical failure buckets.

Batasan Implementasi Lab vs Produksi

Kesenjangan arsitektur yang wajib disesuaikan di skala enterprise.

BATASAN LAB INI

• Penyimpanan in-memory hilang saat instance restart. • Simulasi kompresi waktu (menit mewakili hari). • Evaluasi latency boolean via closure bukan histogram penuh. • Rule berbagi jendela tracker global.

ADAPTASI PRODUKSI

• Gunakan time-series database (Prometheus / VictoriaMetrics). • Terapkan jendela rolling 28 hari (konsistensi akhir pekan). • Manfaatkan prometheus quantile histogram untuk P95/P99. • Simpan status audit freeze di database persisten.

Production deployments require distributed time-series storage and full latency quantiles.

Anti-Patterns & Kesalahan Umum SRE

Jebakan implementasi keandalan yang sering merugikan organisasi.

KESALAHAN METRIK & TARGET

• Menetapkan SLO 100% yang mustahil dicapai. • Menggunakan latency rata-rata (mean) yang menyembunyikan tail latency p99. • Menggunakan metrik hardware CPU/RAM sebagai indikator keandalan layanan.

KESALAHAN OPERASIONAL

• Menjalankan error budget tanpa kebijakan penegakan freeze (hanya menjadi pajangan dasbor). • Membunyikan pager untuk spike sesaat akibat single-window alerting.

An error budget without an enforced release freeze policy is merely a decorative dashboard.

Checklist Implementasi SLO & Error Budget

Panduan verifikasi langkah demi langkah untuk tim pengembang dan SRE.

DESAIN & INDIKATOR

[ ] Tentukan SLI berbasis user experience (rasio good/total). [ ] Tetapkan target SLO realistis (≤ 99.9%, bukan 100%). [ ] Definisikan batasan ambang latency via predikat closure. [ ] Terapkan klasifikasi SLO berbeda sesuai kritisitas bisnis.

ALERTING & KEBIJAKAN

[ ] Bangun mekanisme alerting multi-window multi-burn-rate. [ ] Bedakan severity Page (14.4x) vs Ticket (6.0x). [ ] Tegakkan kebijakan otomatis CanDeploy = false saat budget habis. [ ] Jadwalkan evaluasi berkala kapasitas error budget tiap kuartal.

Enforce quantitative SLIs, dual-window burn rates, and automated release policies.

SOFTWARE ENGINEERING LAB | LAB 24

SLO, SLI, and Error Budget Implementation

SLIs measure customer experience; error budgets allocate risk for engineering innovation; dual-window burn rate alerts catch real outages without waking on-call for transient noise.

> • Verifikasi Otomatis: 6/6 PASS (go test -race ./...) • Kunci Arsitektur: WindowTracker sync.RWMutex, Defensive Zero-Traffic Guard, Dual-Window Burn Rate Engine • Repositori: github.com/lukman-ss/software-engineering-lab/tree/main/labs/24-slo-sli-error-budget • Komitmen Terverifikasi: ffb1d0c4

Karya: Lukman / lukman-ss • Software Engineering Lab Seri 24